

07 实战例题

目标:7 道典型题,展示从输入到提交完整流程,把 00-06 所有知识串起来。

平台:每题注明 LeetCode / 牛客。

风格:每题"题目 → 思路 → Java 完整代码 → C++ 对照 → 提交要点",完整可提交,不是伪代码。

写在前面:怎么看这一篇

这一章是整份速成文档的收官篇。前面 00-06 把语法、容器、工具类、模板、坑点全拆散了讲,这一篇把它们按 7 道具体题目重新组装一遍。

使用方法:

1. 每题按顺序读:题 1 → 题 7 难度递增,先看"思路"段,在脑中走一遍,再翻"Java 完整代码"对照。
2. 代码完整可抄:所有 Java 代码直接复制就能提交(class Solution / class Main 形式都齐),不是伪代码。本地测试的 main 方法注释了"提交时删掉",按提示删掉就能上 LeetCode。
3. C++ 对照是给你"找不同"用的——如果 C++ 写法很熟,看一眼 Java 翻译就能秒懂。
4. 提交要点单独列了"这题最容易踩的坑",提交前必看。
C++ 选手最常踩的坑(温习一下,见 06):
 - Integer 用 == 比较 → 永远用 equals
 - Stack 类 → 永远用 ArrayDeque 替代
 - 字符串不可变 → 改用 char[] 或 StringBuilder
 - LeetCode 模式不写 throws IOException,ACM 模式必写

题 1:A+B(牛客,基础 I/O + ACM 模式 + 多组输入)

平台:牛客 OJ

难度:(入门)

涉及知识:01 输入输出(StreamTokenizer / Scanner / PrintWriter / throws IOException)

题目描述

输入第一行一个整数 T,表示测试组数。接下来 T 行,每行两个整数 a b,输出 a + b。

输入示例:

```
3
1 2
3 4
5 6
```

输出示例:

```
3
7
11
```

思路

这就是牛客最经典的入门题。读完 T 后,用 for 循环读 T 组数据,每组算加法。

关键点:

- 多组输入:用 for (int t = 0; t < T; t++) 包住
- 大输出:StringBuilder 拼好,一次性 PrintWriter 输出
- 快速读入:StreamTokenizer 比 Scanner 快 5-10 倍(数据量大时必用)
- 异常声明:main 必须 throws IOException

Java 完整代码(ACM 模式,StreamTokenizer 版)

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        in.nextToken();
        int T = (int) in.nval;      // 第一行:测试组数

        StringBuilder sb = new StringBuilder();
        for (int t = 0; t < T; t++) {
            in.nextToken();
            int a = (int) in.nval;
            in.nextToken();
            int b = (int) in.nval;
            sb.append(a + b).append("\n");
        }

        out.print(sb);
        out.flush();
    }
}
```

Java 完整代码(Scanner 版,数据量小时用)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        StringBuilder sb = new StringBuilder();
        for (int t = 0; t < T; t++) {
            int a = sc.nextInt();
            int b = sc.nextInt();
            sb.append(a + b).append("\n");
        }
        System.out.print(sb);
    }
}
```



```
#include <iostream>
int main() {
    int T; std::cin >> T;
    while (T--) {
        int a, b; std::cin >> a >> b;
        std::cout << a + b << "\n";
    }
    return 0;
}
```

提交要点

- 类名必须是 Main:牛客 ACM 模式只认 public class Main,写成 Solution 直接编译失败。
- throws IOException 必加:StreamTokenizer / BufferedReader 都抛 IOException,不写编译报错。C++ 选手第一次写 Java 大概率卡这里。
- 数据量 $\leq 10^4$ 用 Scanner:可读性好,不用记 nval 是 double。
- 数据量 $\geq 10^5$ 用 StreamTokenizer:Scanner 在大数据下慢 5-10 倍,牛客卡常数的题会 TLE。
- out.flush() 别忘:不 flush 可能输出不全。
- 不要写 Scanner 又用 nextLine:nextInt 不会读走换行符,下一次 nextLine 立刻读到空串(坑,见 02 第 2 节)。

题 2:反转字符串(LeetCode 344,字符串基础)

平台:LeetCode 344

难度:

涉及知识:02 字符串不可变 + 04 StringBuilder + 05 双指针

题目描述

编写一个函数,其作用是将输入的字符串反转过来。输入字符串以字符数组 `char[]` 的形式给出。不要给另外的数组分配额外的空间,必须原地修改输入数组,使用 $O(1)$ 的额外空间。

输入:["h","e","l","l","o"]

输出:["o","l","l","e","h"]

思路

最优解法:双指针首尾交换($O(n)$ 时间, $O(1)$ 空间)

两个指针 i 从 0 开始, j 从末尾开始,不断交换 `s[i]` 和 `s[j]`,然后 $i++$ 、 $j--$,直到 $i \geq j$ 。

C++ 选手的肌肉记忆是 `std::reverse`,Java 没有 `std::reverse`,得手写这个双指针。LeetCode 题目给的是 `char[]` 不是 `String`,关键原因就是 `String` 不可变(见 02 第二节)——必须给可变的 `char[]` 才能原地改。

Java 完整代码(LeetCode 模式,双指针原地交换)



```

class Solution {
    public void reverseString(char[] s) {
        int i = 0, j = s.length - 1;
        while (i < j) {
            char tmp = s[i];
            s[i] = s[j];
            s[j] = tmp;
            i++;
            j--;
        }
    }

    // ===== 本地测试用,LeetCode 提交时删掉 =====
    public static void main(String[] args) {
        char[] s = {'h', 'e', 'l', 'l', 'o'};
        new Solution().reverseString(s);
        System.out.println(new String(s)); // olleh
    }
}

```

对照版:用 `StringBuilder.reverse()` (不满足 $O(1)$ 空间,仅作 API 演示)

```

class Solution {
    public void reverseString(char[] s) {
        // StringBuilder 有 reverse() 方法,刷题偷懒可抄
        // 但这会创建新 StringBuilder,空间  $O(n)$ ,LeetCode 344 提交能过但不是"最优"
        StringBuilder sb = new StringBuilder(new String(s));
        sb.reverse();
        char[] reversed = sb.toString().toCharArray();
        System.arraycopy(reversed, 0, s, 0, s.length);
    }
}

```

C++ 对照

```

#include <vector>
#include <algorithm>
class Solution {
public:
    void reverseString(std::vector<char>& s) {
        // C++ 一行搞定
        std::reverse(s.begin(), s.end());

        // 或者手写双指针(跟 Java 版几乎一样)
        // int i = 0, j = s.size() - 1;
        // while (i < j) { std::swap(s[i++], s[j--]); }
    }
};

```

提交要点



- 原地修改必须用 `char[]`:Java 的 `String` 不可变(02 第二节),`s[0] = 'X'` 直接编译报错。LeetCode 344 题面把入参设计成 `char[]` 就是这个原因。
 - 不要用 `StringBuilder.reverse()` 当“正解”:虽然能过,但创建了额外的 `StringBuilder` 对象,空间复杂度 $O(n)$,不是“原地 $O(1)$ ”。面试时这么写会被追问。
 - 交换用 `tmp` 中间变量:Java 没有 `std::swap`,老老实实写 3 行交换。C++ 选手别用 `(s[i], s[j]) = (s[j], s[i])`(Java 没有 `tuple swap` 语法)。
 - `s.length` 是字段不是方法:`s.length` 无括号(02 第三节),写 `s.length()` 编译报错。
 - LeetCode 模式不写 `throws IOException`:平台帮你 `try-catch` 了,跟 ACM 模式不一样。
-

题 3:反转链表(LeetCode 206,指针操作)

平台:LeetCode 206

难度:

涉及知识:02 类与对象(`ListNode`) + 05 双指针(三指针)

题目描述

给你单链表的头节点 `head`,请你反转链表,并返回反转后的链表。

输入:1 -> 2 -> 3 -> 4 -> 5

输出:5 -> 4 -> 3 -> 2 -> 1

思路

最优解法:三指针迭代($O(n)$ 时间, $O(1)$ 空间)

三个指针 `prev` / `cur` / `nxt`:

- `cur` 指向当前节点
- `nxt = cur.next`(提前保存,否则改了 `cur.next` 就找不到了)
- `cur.next = prev`(把当前节点的 `next` 反向)
- 三个指针都前移:`prev = cur; cur = nxt`

最后 `prev` 指向新头节点。

为什么需要第三个指针 `nxt`? 改了 `cur.next` 之后,原链表就断了,必须先把 `cur.next` 存到 `nxt`,否则下一步就找不到原链表的下一个节点了。C++ 选手第一次写链表反转大概率栽这个点。

Java 完整代码(LeetCode 模式,三指针迭代)



```

class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }    // 构造器(★ 必背)
}

class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;    // 新链表的尾,初始 null
        ListNode cur = head;
        while (cur != null) {
            ListNode nxt = cur.next; // 1. 先存后继
            cur.next = prev;        // 2. 反向指针
            prev = cur;             // 3. prev 前移
            cur = nxt;              // 4. cur 前移
        }
        return prev;               // prev 就是新头
    }

    // ===== 本地测试用,LeetCode 提交时删掉 =====
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        ListNode head = buildList(arr);
        head = new Solution().reverseList(head);
        printList(head);           // [5,4,3,2,1]
    }

    static ListNode buildList(int[] arr) {
        ListNode dummy = new ListNode(0);
        ListNode cur = dummy;
        for (int x : arr) {
            cur.next = new ListNode(x);
            cur = cur.next;
        }
        return dummy.next;
    }

    static void printList(ListNode head) {
        StringBuilder sb = new StringBuilder("");
        for (ListNode p = head; p != null; p = p.next) {
            if (sb.length() > 1) sb.append(",");
            sb.append(p.val);
        }
        sb.append("\n");
        System.out.println(sb);
    }
}

```

C++ 对照



```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v = 0) : val(v), next(nullptr) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    for (ListNode* cur = head; cur != nullptr; ) {
        ListNode* nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}

```

提交要点

- ListNode 类放在 Solution 同文件:LeetCode 提交时把 class ListNode 放在 class Solution 上面(02 第六节),不要嵌套在 Solution 里(老版要 static 修饰,新版可省略,推荐放外面)。
- ListNode 构造器只写一个参数就够:ListNode(int v) { val = v; }。next 不用初始化Java 引用类型字段默认是 null(02 第三节),C++ 选手别画蛇添足写 next = null。
- 哑节点(dummy)简化链表的"头节点"边界:本地的 buildList 用了 dummy = new ListNode(0),免去判断"头节点是不是第一个"的麻烦,面试手写链表推荐都用。
- 三指针顺序不能乱:nxt 必须在改 cur.next 之前保存。很多初学者写成 cur.next = prev; nxt = cur.next; 这是 bug——cur.next 已经被改了。
- 没有 struct 关键字:Java 只有 class,即使行为跟 C++ struct 一样,也得写 class ListNode。
- 访问成员用 . 不是 ->;Java 引用就是"指向对象的句柄",p.next 跟 C++ p->next 等价(02 第一节)。

题 4:二叉树层序遍历(LeetCode 102,ArrayDeque + BFS)

平台:LeetCode 102

难度:

涉及知识:02 类与对象(TreeNode) + 03 ArrayDeque(队列) + 05 BFS

题目描述

给你二叉树的根节点 root,返回其节点值的层序遍历。即逐层地,从左到右访问所有节点。

输入:

```

    3
   /\
  9 20
 /\
15 7

```

输出:[[3],[9,20],[15,7]]

思路



最优解法:BFS + 每层提前取 size($O(n)$ 时间, $O(n)$ 空间)

经典 BFS 套路:

1. 队列初始化,放入 root

2. while 队列不空:

- `int sz = q.size();`(关键!每层 size 在 for 外提前取)
- `for i = 0; i < sz; i++:`
- 弹出一个节点,加入当前层结果
- 把左右孩子(非空)入队
- 把当前层结果加入总结果

为什么 sz 必须在 for 外取? 因为 for 循环里会不断 offer 左右孩子,队列大小边遍历边变。如果在 for 条件里写 `i < q.size()`,就变成"无限循环"了。这个坑 90% 初学者会踩。

C++ 选手用 C++ `std::queue` 时也是一样的套路,完全平移。

Java 完整代码(LeetCode 模式,BFS + ArrayDeque)



```

import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}

class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res; // 空树直接返回
        ArrayDeque<TreeNode> q = new ArrayDeque<>(); // ★ 用 ArrayDeque 不用 LinkedList
        q.offer(root);
        while (!q.isEmpty()) {
            int sz = q.size(); // ★ 关键:在 for 外取 size
            List<Integer> level = new ArrayList<>(sz);
            for (int i = 0; i < sz; i++) {
                TreeNode node = q.poll();
                level.add(node.val);
                if (node.left != null) q.offer(node.left);
                if (node.right != null) q.offer(node.right);
            }
            res.add(level);
        }
        return res;
    }

    // ===== 本地测试用,LeetCode 提交时删掉 =====
    public static void main(String[] args) {
        Integer[] arr = {3, 9, 20, null, null, 15, 7}; // ★ 必须用 Integer(不是 int),因为有 null
        TreeNode root = buildTree(arr);
        System.out.println(new Solution().levelOrder(root));
    }

    // LeetCode 标准:层序数组 → 二叉树(★ 必背,本地测试必用)
    static TreeNode buildTree(Integer[] arr) {
        if (arr == null || arr.length == 0 || arr[0] == null) return null;
        TreeNode root = new TreeNode(arr[0]);
        ArrayDeque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        int i = 1;
        while (!q.isEmpty() && i < arr.length) {
            TreeNode parent = q.poll();
            if (i < arr.length && arr[i] != null) {
                parent.left = new TreeNode(arr[i]);
                q.offer(parent.left);
            }
            i++;
            if (i < arr.length && arr[i] != null) {
                parent.right = new TreeNode(arr[i]);
                q.offer(parent.right);
            }
            i++;
        }
    }
}

```



```

    }
    return root;
}
}

```

C++ 对照

```

#include <vector>
#include <queue>
struct TreeNode {
    int val;
    TreeNode* left, * right;
    TreeNode(int v = 0) : val(v), left(nullptr), right(nullptr) {}
};

std::vector<std::vector<int>>> levelOrder(TreeNode* root) {
    std::vector<std::vector<int>>> res;
    if (!root) return res;
    std::queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();          // 同样要在 for 外取
        std::vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}

```

提交要点

- 每层 size 必须在 for 外取:for (int i = 0; i < q.size(); i++) 看起来人畜无害,实际是死循环。这是 BFS 模板的最大坑。
- 用 ArrayDeque 不用 LinkedList:Java 队列/栈 99% 场景用 ArrayDeque,性能比 LinkedList 好 2-3 倍(03 第五节)。也别用 java.util.Stack(03 简讲容器第 1 条)。
- buildTree(Integer[] arr) 是 LeetCode 标配:本地测试把层序数组还原成树,LeetCode 平台自动给的是 TreeNode,你本地要自己构造。这段代码背下来,以后刷树题都用。
- 数组类型用 Integer 不能用 int:Integer[] arr = {3, 9, 20, null, null, 15, 7} 可以写 null,int[] arr = {...} 写不了 null(因为 int 是基本类型不是引用类型)。
- null 判空用 if (node.left != null):Java 用 null 标记"空"(int[] 默认全 0,Integer[] 默认全 null,见 02 第三节)。C++ 用 nullptr 或 NULL。
- q.offer(x) 还是 q.add(x)? 两者效果一样,offer 返回 boolean,add 容量满时抛异常。队列场景用 offer(语义更准),add 是 Collection 接口的通用方法。



题 5:两数之和(LeetCode 1,HashMap 单遍)

平台:LeetCode 1

难度:

涉及知识:03 HashMap + 04 getOrDefault / containsKey

题目描述

给定一个整数数组 `nums` 和一个整数目标值 `target`,请你在该数组中找出和为目标值的那两个整数,并返回它们的数组下标。

输入:`nums = [2,7,11,15]`, `target = 9`

输出:`[0,1]`(因为 `nums[0] + nums[1] = 9`)

思路

最优解法:HashMap 单遍遍历($O(n)$ 时间, $O(n)$ 空间)

核心思想:边遍历边查——遍历到 `nums[i]` 时,看 `target - nums[i]`(记作 `need`)之前有没有出现过。

- 用一个 `HashMap<Integer, Integer> idx`,`key = 数组元素值`,`value = 元素下标`。
- 遍历每个 `nums[i]`:
- 算 `need = target - nums[i]`
- 如果 `idx.containsKey(need)` → 找到答案 `[idx.get(need), i]`
- 否则 `idx.put(nums[i], i)`

为什么单遍就够? 因为我们先查再存:遍历到 `i` 时,`idx` 里存的是 `0..i-1` 的元素,正好对应"之前出现过的数"。不需要先存一遍再查。

C++ 选手的肌肉记忆是双指针排序版($O(n \log n)$),LeetCode 1 因为要返回原始下标不能用排序(会乱),所以 HashMap 单遍是唯一最优解。

Java 完整代码(LeetCode 模式,HashMap 单遍)



```

import java.util.*;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> idx = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (idx.containsKey(need)) { // ★ containsKey,不是 get != null
                return new int[]{idx.get(need), i};
            }
            idx.put(nums[i], i);
        }
        return new int[0]; // 题目保证有解,实际不会走到
    }

    // ===== 本地测试用,LeetCode 提交时删掉 =====
    public static void main(String[] args) {
        int[] nums = {2, 7, 11, 15};
        int target = 9;
        int[] res = new Solution().twoSum(nums, target);
        System.out.println(Arrays.toString(res)); // [0, 1]
    }
}

```

对照版:用 `getOrDefault` 改成"两数和"计数题(LeetCode 167 / 剑指 Offer)

```

// LeetCode 167:两数之和 II(有序数组,要求下标从 1 开始)
// 严格说这题用双指针,这里用 HashMap 是另一种写法
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> idx = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (idx.containsKey(need)) return new int[]{idx.get(need), i};
            idx.put(nums[i], i);
        }
        return new int[0];
    }
}

```

C++ 对照



```
#include <vector>
#include <unordered_map>
class Solution {
public:
    std::vector<int> twoSum(std::vector<int>& nums, int target) {
        std::unordered_map<int, int> idx;
        for (int i = 0; i < nums.size(); i++) {
            int need = target - nums[i];
            if (idx.count(need)) { // count > 0 等价于 containsKey
                return {idx[need], i};
            }
            idx[nums[i]] = i;
        }
        return {};
    }
};
```

提交要点

- 必须用 `containsKey`, 不要用 `get(need) != null`: 这是 LeetCode 1 最大的坑。`nums[i]` 可能为 0, `Map.get(0)` 返回的是包装类 `Integer 0`, 跟 `null` 比较结果竟然是 `false`, 不会误判; 但如果题目是 "`nums[i]` 可能为 0 改成自定义对象", 或者键类型语义不同时, 用 `get != null` 容易翻车。永远用 `containsKey` 是 Java 哈希表的铁律 (C++ 没有这个坑, 因为 `idx[need]` 不存在时会插入默认值, 反而是 C++ 容易踩)。
- 不能先 `get` 再判断: `int x = map.get(need); if (x != null) ...` 看着聪明, 实际有 "键不存在时返回 `null`" 和 "键存在但值为 `null`" 的歧义。`containsKey` 没有这个问题。
- 单遍够, 不需要两遍: 很多人本能写两遍 (第一遍存, 第二遍查), 其实边遍历边查就行, 先查再存。LeetCode 1 的官方题解就是这个。
- C++ 选手的双指针排序版不行: C++ 经典写法是排序后双指针, 但 LeetCode 1 要求返回原始下标, 排序会乱序。HashMap 是唯一最优解。
- 返回 `int[]` 不是 `List<Integer>`: 看题目要求, LeetCode 1 是 `int[]`, 别用 `ArrayList<Integer>` (虽然能过, 但不是题目要的返回类型)。

题 6: 滑动窗口最大值 (LeetCode 239, ArrayDeque 单调队列)

平台: LeetCode 239

难度:

涉及知识: 03 ArrayDeque (双端队列) + 05 单调队列 (单调递减)

题目描述

给你一个整数数组 `nums`, 有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。返回滑动窗口中的最大值。

输入: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

输出: `[3,3,5,5,6,7]`

思路

最优解法: 单调队列 (降序) ($O(n)$ 时间, $O(k)$ 空间)

核心思想: 维护一个 "队首最大" 的双端队列, 队列里存的是数组下标 (不是值), 下标对应的值单调递减。



每个新元素 `nums[i]` 进来时,做三件事:

1. 移除窗口外:队首下标如果 $< i - k + 1$,说明已经滑出窗口了,pollFirst
2. 维护单调性:从队尾开始,把所有比 `nums[i]` 小的下标 pollLast 掉(它们永远不可能是最大值了)
3. 入队:offerLast(i)
4. 记录答案: $i \geq k - 1$ 时,`nums[队首下标]` 就是当前窗口最大值

为什么存下标不存值? 因为要判断"队首是否已经滑出窗口",需要下标做差。存值的话,不知道这个值原来在哪个位置,无法判断是否还在窗口内。

为什么单调递减? 因为我们只关心"最大值",队首就是最大。如果新元素比队尾大,队尾那个小元素就再也轮不到当最大值了,直接踢掉。

Java 完整代码(LeetCode 模式,单调队列)

```
import java.util.*;

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        if (nums == null || nums.length == 0) return new int[0];
        int n = nums.length;
        int[] res = new int[n - k + 1]; // 答案数组
        ArrayDeque<Integer> dq = new ArrayDeque<>(); // ★ 存下标(不是值),单调递减

        for (int i = 0; i < n; i++) {
            // 1. 移除窗口外的下标(队首如果  $< i - k + 1$ ,说明滑出去了)
            while (!dq.isEmpty() && dq.peekFirst() < i - k + 1) {
                dq.pollFirst();
            }
            // 2. 维护单调性:从队尾弹出所有比 nums[i] 小的下标
            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) {
                dq.pollLast();
            }
            // 3. 当前下标入队
            dq.offerLast(i);
            // 4. 窗口形成后( $i \geq k - 1$ ),队首就是当前窗口最大值
            if (i >= k - 1) {
                res[i - k + 1] = nums[dq.peekFirst()];
            }
        }
        return res;
    }

    // ===== 本地测试用,LeetCode 提交时删掉 =====
    public static void main(String[] args) {
        int[] nums = {1, 3, -1, -3, 5, 3, 6, 7};
        int k = 3;
        int[] res = new Solution().maxSlidingWindow(nums, k);
        System.out.println(Arrays.toString(res)); // [3, 3, 5, 5, 6, 7]
    }
}
```

C++ 对照



```
#include <vector>
#include <deque>
class Solution {
public:
    std::vector<int> maxSlidingWindow(std::vector<int>& nums, int k) {
        std::vector<int> res;
        std::deque<int> dq;           // 存下标,单调递减
        for (int i = 0; i < nums.size(); i++) {
            while (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
            while (!dq.empty() && nums[dq.back()] < nums[i]) dq.pop_back();
            dq.push_back(i);
            if (i >= k - 1) res.push_back(nums[dq.front()]);
        }
        return res;
    }
};
```

提交要点

- 队列存的是下标,不是值:这是这题最关键的设计。存值的话无法判断队首是否在窗口内,要么超时,要么写错。
- ArrayDeque 不能存 null:dq.offerLast(null) 直接抛 NullPointerException(03 第五节坑点 1)。如果题目的数组可能有"无效位置"概念,得改用 LinkedList。
- 队首判断用 $i - k + 1$ 不是 $\leq i - k$:窗口是 $[i - k + 1, i]$,长度 k 。比如 $i = 2, k = 3$,窗口是 $[0, 2]$,下标 0 在窗口内。所以下标 < 0 才算滑出, $i - k + 1$ 即 < 0 ,pollFirst。
- 判断"比当前小"是从队尾 pollLast:不是从队首。从队首 poll 会破坏"队首最大"的性质。
- 不要写成大顶堆 PriorityQueue:用堆的话,每次要"清理堆顶过期元素",需要再记一个 map,代码长一倍。单调队列是这题的最优解。
- C++ 选手的 priority_queue 习惯改掉:这题是少数 deque 优于 priority_queue 的场景。面试被问到单调队列,要能讲清楚"为什么 deque 够用,不用堆"。

题 7:跳跃游戏(LeetCode 55,贪心)

平台:LeetCode 55

难度:

涉及知识:05 贪心(局部最优 $\rightarrow$ 全局最优)

题目描述

给定一个非负整数数组 nums,你最初位于数组的第一个下标。数组中的每个元素代表你在该位置可以跳跃的最大长度。判断你是否能够到达最后一个下标。

输入:nums = [2,3,1,1,4] $\rightarrow$ 输出 true

输入:nums = [3,2,1,0,4] $\rightarrow$ 输出 false

思路

最优解法:贪心,维护"最远可达下标"(O(n) 时间,O(1) 空间)

核心思想:不用真模拟跳,只关心"最远能到哪"。

维护一个变量 maxReach,表示当前能跳到的最远下标。遍历数组,每到一个位置 i:



- 如果 $i > \text{maxReach}$ → 这个位置根本到不了,直接返回 false
- 否则,更新 $\text{maxReach} = \max(\text{maxReach}, i + \text{nums}[i])$ (从 i 出发能跳到 $i + \text{nums}[i]$)

遍历结束, $\text{maxReach} \geq n - 1$ 就返回 true。

贪心的正确性:局部最优(每步能到最远)→ 全局最优(能到末尾)。不需要 DP, $O(n)$ 一次遍历搞定。

为什么不需要 DP? 位置 i 能否到达,只跟"之前能到的最远下标"有关,跟"怎么到 i "无关。这种"只关心最值不关心路径"的问题,贪心比 DP 更直接。

Java 完整代码(LeetCode 模式,贪心)

```
class Solution {
    public boolean canJump(int[] nums) {
        int maxReach = 0;           // 最远可达下标
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            if (i > maxReach) return false;    // ★ i 到不了,直接 false
            maxReach = Math.max(maxReach, i + nums[i]);
            if (maxReach >= n - 1) return true; // 已经能到末尾,提前返回
        }
        return true;
    }

    // ===== 本地测试用,LeetCode 提交时删掉 =====
    public static void main(String[] args) {
        Solution sol = new Solution();
        System.out.println(sol.canJump(new int[]{2, 3, 1, 1, 4})); // true
        System.out.println(sol.canJump(new int[]{3, 2, 1, 0, 4})); // false
    }
}
```

C++ 对照

```
#include <vector>
#include <algorithm>
class Solution {
public:
    bool canJump(std::vector<int>& nums) {
        int maxReach = 0;
        int n = nums.size();
        for (int i = 0; i < n; i++) {
            if (i > maxReach) return false;
            maxReach = std::max(maxReach, i + nums[i]);
            if (maxReach >= n - 1) return true;
        }
        return true;
    }
};
```

提交要点

• 贪心不需要 DP:这题很多人本能想用 DP($\text{dp}[i]$ = 能否到达 i), $O(n^2)$ 也能过,但 $O(n)$ 贪心才是最优解。面试写出 DP 要被追问"能不能优化",直接写贪心一步到位。



- 关键判断 `if (i > maxReach)`:这是贪心的"剪枝"——如果当前位置都到不了,后面的位置更到不了。没有这一行就是错的(虽然可能不会触发返回 `false` 的情况,但不严谨)。
- `i + nums[i]` 是从 `i` 出发能到最远的位置:C++ 选手要注意 `nums[i]` 是"最大跳跃长度",可以跳 1 到 `nums[i]` 任意长度,所以最远能到 `i + nums[i]`。
- 不要用 `HashSet` 记访问过的位置:这是路径搜索的思路,完全错。这题根本不关心路径,只关心"最远能到哪"。
- 进阶题 LeetCode 45(跳跃游戏 II,求最小跳跃次数)也用贪心,思路是"在当前可跳范围内,选能跳最远的下一跳起点"。做完 55 再去做 45 顺手。
- `Math.max` 是 `int` 重载:`maxReach`、`i`、`nums[i]` 都是 `int`,自动选 `int` 版本,不需要转 `long`。如果数据可能超过 `int`(比如 `nums[i]` 接近 `Integer.MAX_VALUE`),用 `long` 重载要 `Math.max((long)maxReach, (long)i + nums[i])`。

7 道题总览对照表(回看 00 总览表)

目的:做完 7 道题后,回到 00 总览表,看自己用了哪些 C++ → Java 翻译点。

#	题目	平台	难度	核心 API / 容器	对应 C++	涉及章节
1	A+B(多组输入)	牛客	★	<code>StreamTokenizer</code> / <code>PrintWriter</code> / <code>Scanner</code>	<code>cin</code> / <code>cout</code>	01
2	反转字符串	LeetCode 344	★	<code>char[]</code> / <code>StringBuilder.reverse()</code>	<code>std::string</code> / <code>std::reverse</code>	02 + 04 + 05
3	反转链表	LeetCode 206	★☆	<code>class ListNode</code> / 三指针	<code>struct ListNode</code> / 三指针	02 + 05
4	二叉树层序遍历	LeetCode 102	★★	<code>class TreeNode</code> / <code>ArrayDeque</code> / BFS	<code>struct TreeNode</code> / <code>std::queue</code>	02 + 03 + 05
5	两数之和	LeetCode 1	★	<code>HashMap</code> / <code>containsKey</code>	<code>unordered_map</code> / <code>count</code>	03 + 04
6	滑动窗口最大值	LeetCode 239	★★★★	<code>ArrayDeque</code> 单调队列(下标)	<code>deque</code>	03 + 05
7	跳跃游戏	LeetCode 55	★★	贪心 / <code>Math.max</code>	贪心 / <code>std::max</code>	05

7 道题覆盖了 7 个维度:

- I/O(题 1)/ 字符串(题 2)/ 链表(题 3)/ 树(题 4)/ 哈希(题 5)/ 单调队列(题 6)/ 贪心(题 7)
- 刷完这 7 道题,Java 刷题的"语言切换"基本过关。

刷题节奏建议

>

- 第一遍(本节配速成):看思路 → 抄代码 → 改 1-2 个变量名验证理解
- 第二遍(独立刷):不看思路,自己 5-10 分钟内写完,卡住再看
- 第三遍(拓展):去 LeetCode 按标签(链表 / 树 / 哈希 / 单调队列 / 贪心)各刷 5-10 道
- 关键:每题做完对照 00 总览表,看自己用了哪些容器/API,加深印象
- 完成度自检:7 道题如果都能独立写完,LeetCode 中等题通过率 70%+ 的目标基本达成



- 遇到不熟 API:先翻速查表(Java刷题速成/速查表.md),再翻对应章节,不要从 00 一路翻到 07

下一步:打开 速查表.md(单页 CheatSheet),把 8 篇内容浓缩成一页,做题时反复翻 > 翻文档。速查表是长期工具,不是"读一遍就丢"的东西。

